Race Conditions and synchronized

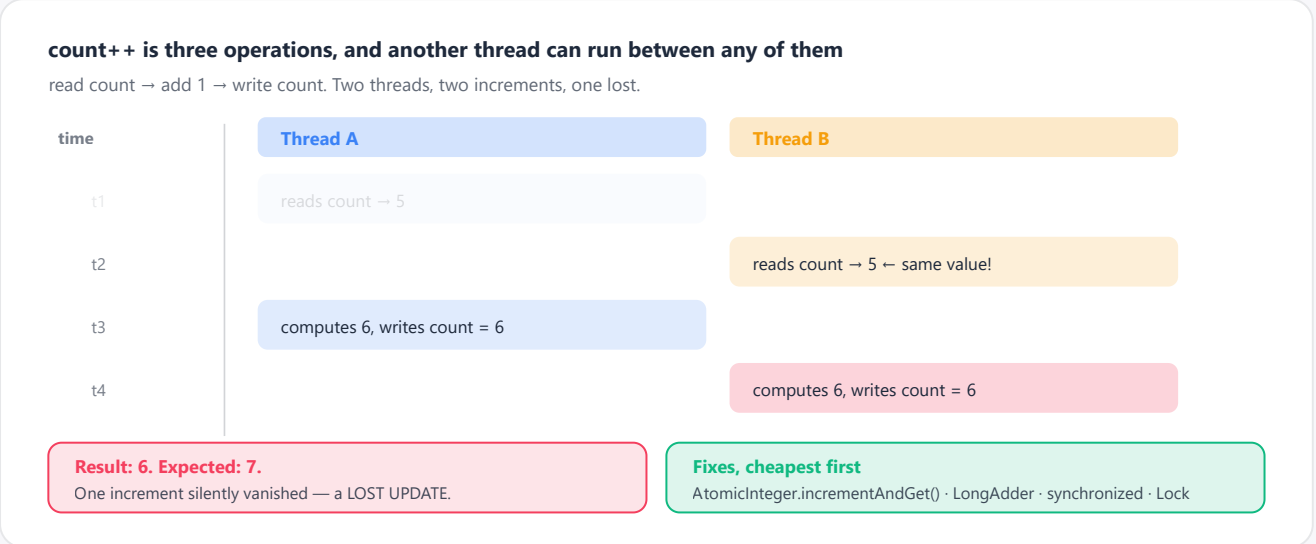
9 min read · Updated 28 Jul 2026 · Topic 37 of 65

Why it matters

A race condition is the defining concurrency bug: it passes every test, works in staging, and corrupts data in production under load. Understanding why `count++` is unsafe — and why the fix is not what most people first reach for — is the foundation for everything else in this chapter.

The concept, simply

A **race condition** is when the result depends on the unpredictable timing of threads. The canonical example is an increment, which looks atomic and is not:



count++ is read, add, write. Interleave two threads across those three steps and one update silently disappears — a lost update. Nothing throws.

`synchronized` fixes it by giving exactly one thread at a time access to the critical section. Every Java object has a **monitor** (an intrinsic lock); entering a `synchronized` block acquires it and leaving releases it — including when an exception unwinds.

`synchronized` provides **two** guarantees, and forgetting the second one causes subtle bugs:

- **Mutual exclusion** — one thread in the critical section at a time.
- **Visibility** — releasing a monitor flushes changes to main memory, and acquiring it invalidates the cached copy, so the next thread sees everything the previous one did. This is the *happens-before* relationship of the next topic.

What object is locked matters more than people expect:

Form	Locks
`synchronized void m()`	`this` — the instance
`static synchronized void m()`	The `Class` object — **a different lock**
`synchronized (obj) { }`	Whatever `obj` is

That third row is why an instance method and a static method on the same class can run *simultaneously*: they hold different monitors. It is a favourite interview question and a real source of bugs.

Intrinsic locks are reentrant: a thread already holding a monitor can acquire it again without deadlocking itself. Without that, a `synchronized` method calling another `synchronized` method on the same object would hang instantly.

Code examples

JAVA

Copy

```java
// ✗ Race — count++ is three operations.
class Counter {
    private int count;
    public void increment() { count++; }
    public int get() { return count; }
}
```

```java
// Two threads × 10,000 increments → typically ~13,000, never reliably 20,
// ✓ Correct, and note get() must be synchronized too — otherwise a reader
//   may see a stale cached value even though writes are exclusive.
class SafeCounter {
    private int count;
    public synchronized void increment() { count++; }
    public synchronized int get() { return count; }
}

// ✓ Better for this specific case — no lock at all (last topic).
class AtomicCounter {
    private final AtomicInteger count = new AtomicInteger();
    public void increment() { count.incrementAndGet(); }
    public int get() { return count.get(); }
}
```

Lock scope, which is where most of the performance is won or lost:

JAVA Copy

```java
// ✗ Locks for the whole method, including the expensive part that needs
//   no protection at all. Every other thread waits on the network call.
public synchronized void process(Order order) {
    Result r = callExternalService(order);   // 200ms, no shared state
    this.total += r.amount();                // the only line that races
}

// ✓ Lock only the critical section.
public void process(Order order) {
    Result r = callExternalService(order);   // outside the lock
    synchronized (this) {
        this.total += r.amount();
    }
}
// Never do I/O or call unknown code while holding a lock — that is how a
// slow dependency turns into an application-wide stall.
```

The private lock object, which is a genuine design improvement:

JAVA Copy

```
//   including deadlocking you — and you cannot prevent it.
public class Registry {
    public synchronized void add(String k) { … }
}

// ✓ A private lock nobody outside can acquire.
public class Registry {
    private final Object lock = new Object();      // final: never reassign
    public void add(String k) {
        synchronized (lock) { … }
    }
}
```

And the two locks that are not one lock:

JAVA Copy

```
class Mixed {
    private static int staticCount;
    private int instanceCount;

    public synchronized void incInstance() { instanceCount++; }      // lo
    public static synchronized void incStatic() { staticCount++; }   // lo
}
// These two methods do NOT exclude each other — different monitors. If bo
// touched the same shared state, that would be a race despite both being
// synchronized. A classic "spot the bug" question.

// Also: never synchronize on a boxed value or a String literal — they are
// pooled and SHARED across your whole application:
// synchronized ("lock") { }              ✗ every class using "lock" share
// synchronized (Integer.valueOf(1)) { }  ✗ the Integer cache is global
```

Under the hood

In bytecode, a `synchronized` block compiles to `monitorenter` and `monitorexit`, with a synthetic exception handler guaranteeing the exit even when the block throws. A

HotSpot optimises locks aggressively, which is why the old "always use `ReentrantLock`, `synchronized` is slow" advice is obsolete. An uncontended monitor uses a lightweight CAS on the object header rather than an OS mutex; escape analysis can remove locking on an object that never escapes a method (*lock elision*); and adjacent synchronized blocks on the same object may be merged (*lock coarsening*). An uncontended `synchronized` block costs very little. *Biased locking*, which optimised the repeatedly-same-thread case, was deprecated in Java 15 and removed later — worth knowing, because older material still describes it as current.

Contention is what costs, not synchronisation itself. When a thread cannot get the monitor it parks, which means a context switch and a trip through the scheduler. The fix is almost never a faster lock — it is a smaller critical section, a different data structure, or no sharing at all.

The cheapest concurrency strategy is confinement. A local variable cannot be shared (Chapter 1), so it needs no synchronisation. Immutable objects likewise. Reaching for a lock should be the third option, after "does this need to be shared?" and "can this be immutable?".

Common mistakes

- **Synchronising the writer but not the reader.** Readers can see stale values; visibility needs both sides.
- **Assuming `count++` is atomic.** It is read-modify-write.
- **Locking too much.** Holding a lock across I/O serialises the whole application.
- **Synchronising on `this` in a public class.** External code can acquire the same lock. Use a private final lock object.
- **Synchronising on a String literal or boxed value.** Pooled and shared JVM-wide.
- **Synchronising on a non-final field.** Reassigning it means threads lock different objects.
- **Confusing instance and static locks.** They are separate monitors.

Interview lens

Q: "Why is `count++` not thread-safe?"

A: It is three operations — read the field, add one, write it back — and another thread can run between any of them. Two threads can both read 5, both compute 6, and both write 6, so one increment is lost. Nothing throws; the count is just quietly wrong, and only under load.

Q: "What does `synchronized` guarantee?"

A: Two things. Mutual exclusion — one thread at a time in the critical section. And visibility — releasing the monitor flushes writes to main memory and acquiring it invalidates cached values, so the next thread sees everything the previous one did. People remember the first and forget the second, which is why they synchronise the writer and leave the reader unsynchronised.

Q: "What object does a `synchronized` method lock?"

A: An instance method locks `this`; a static method locks the `Class` object. They are different monitors, so an instance method and a static method on the same class can run at the same time — which is a real bug if they touch the same state.

Q: "Why prefer a private lock object over `synchronized` methods?"

A: Because `this` is public. Any code holding a reference to your object can write `synchronized(obj)` and interfere with your locking — hold it too long, or deadlock you — and you have no way to prevent it. A `private final Object lock` is unreachable from outside, so the locking protocol is genuinely yours.

A: Yes. The JVM tracks the owning thread and a hold count, so a thread already holding a monitor can acquire it again. Without that, one `synchronized` method calling another on the same object would deadlock immediately — reentrancy is what makes composition possible.

Q: "Is `synchronized` slow?"

A: Uncontended, barely — HotSpot uses a lightweight CAS on the object header, and can elide locking entirely when escape analysis proves the object never escapes, or coarsen adjacent blocks. The cost is contention: a thread that cannot acquire the monitor parks, which means a context switch. So the fix for slow synchronised code is a smaller critical section or less sharing, not a different lock implementation.

Cheat-card

- Race = result depends on timing. `count++` is **read-modify-write**.
- `synchronized` gives **mutual exclusion + visibility**. Synchronise **readers too**.
- Instance method → locks `this`; static → locks `Class`. Different monitors.
- Prefer a `private final Object lock` over locking `this`.
- Never lock a String literal, a boxed value, or a non-final field.
- Intrinsic locks are **reentrant**. `monitorenter` / `monitorexit` in bytecode.
- **Never do I/O while holding a lock.** Keep critical sections tiny.
- Uncontended locks are cheap (CAS, elision, coarsening). **Biased locking removed in Java 15+.**
- Cheapest strategy: **don't share** — confinement, then immutability, then locks.

QUICK CHECK — no marks, just for you.

1. What TWO guarantees does synchronized provide?

- ◯ Mutual exclusion and VISIBILITY — releasing the monitor flushes writes and acquiring it invalidates cached values
- ◯ Atomicity and immutability
- ◯ Ordering and fairness

Check answer

2. An instance method and a static method on the same class are both synchronized. Can they run at the same time?

- ◯ No, synchronized always serialises the whole class
- ◯ Only if the class is final
- ◯ Yes — the instance method locks 'this' while the static method locks the Class object; they are different monitors
- ◯ No, they share the class's lock

Check answer

3. Why prefer a private final lock object over synchronizing on 'this'?

- ◯ Because 'this' cannot be used in static methods
- ◯ It is faster
- ◯ It allows more threads to enter
- ◯ Because 'this' is public — any external code holding a reference can synchronize on your object and interfere with your locking, including deadlocking you

Previous
Threads: Creation and Lifecycle

Next
volatile and the Java Memory Model

LEARN MORE

ARTICLES & DOCS

- Synchronization — The Java Tutorials
  docs.oracle.com
- Synchronized methods and blocks (JLS §17.1)
  docs.oracle.com

VIDEOS



The Volatile and Synchronized Keywords in Java | Atomic Variables | Java Multithreading | Geekific
Geekific



Java Multithreading: Synchronization, Locks, Executors, Deadlock, CountdownLatch & CompletableFuture
Engineering Digest

Hand-picked references for this lesson.